

Task 1: Create a Basic RESTful API

Use Express.js to implement the following endpoints for managing student records

Please find all the code in the 'app' folder. The database is in 'app\db' folder. Below screenshot demonstrates the setup of express.js and connection to SQLite database.

```
JS app.js U X
Module 12 > Module 12_Week 14_Required Assignment_Patil > app > JS app.js > app.post('/students') callback
1  const express = require('express');
2  const cors = require('cors');
3  const app = express();
4  const port = 3000;
5
6  app.use(cors())
7  app.use(express.json())
8
9  /*** Connect to the database */
10 const sqlite3 = require('sqlite3').verbose()
11 const db = new sqlite3.Database('./db/students.db')
12
106
107 app.listen(port, function(){
108     console.log(`Express app listening on port ${port}!`);
109 })
```

a) GET /students: Retrieve a list of all students.

Below screenshot from app\app.js shows the 'GET' endpoint to retrieve the list of all student records.

```
13 /*** GET endpoint - Retrieve a list of all students. */
14 app.get('/students', function(req, res){
15     db.all('Select * from students order by name ASC', (err, rows) =>{
16         if(err){
17             res.status(404).send('No Student record found!');
18             return;
19         }
20         res.status(200).json(rows);
21     });
22 });
```

b) GET /students/:id: Retrieve a specific student by its ID.

Below screenshot from app\app.js shows the 'GET' endpoint to retrieve the student record by id.

```

23
24  /**** GET endpoint - Retrieve a specific student by its ID. */
25  app.get('/students/:id', function(req, res){
26    db.get('Select * from students where id = ?', [req.params.id], (err, row) =>{
27      if(err || !row){
28        res.status(404).send('Student record not found!');
29        return;
30      }
31      res.status(200).json(row);
32    });
33  });
34

```

c) POST /students: Add a new student.

Below screenshot from app\app.js shows the 'POST' endpoint to insert a record for new student into database.

```

34
35  /**** POST endpoint - Add a new student */
36  app.post('/students', function(req, res){
37    /**** Validate */
38    const newStudentRec = req.body;
39    if(!newStudentRec.name){
40      return res.status(400).json({ error: 'Name is required.' });
41    }
42
43    db.run('Insert into students (name, email, program)
44          VALUES (?, ?, ?)',
45          [
46            req.body.name,
47            req.body.email,
48            req.body.program
49          ],
50          function(err){
51            if(err || this.changes === 0){
52              res.status(400).send('Error creating new student record!');
53              return;
54            }
55            res.status(201).send('Student record created with id: ' + this.lastID);
56          });
57  });
58

```

d) PUT /students/:id: Update an existing student.

Below screenshot from app\app.js shows the 'PUT' endpoint to update a record of existing student by id.

```

58
59  /**** PUT endpoint - Update an existing student. */
60  app.put('/students/:id', function(req,res){
61      /**** Validate */
62      const newStudentRec = req.body;
63      if(!newStudentRec.name){
64          return res.status(400).json({ error: 'Name is required.' });
65      }
66
67      db.run(`UPDATE students SET name = ?, email = ?, program = ? WHERE id = ?`,
68          [
69              req.body.name,
70              req.body.email,
71              req.body.program,
72              req.params.id
73          ],
74          function(err){
75              if(err){
76                  res.status(400).send('Error updating the Student record!');
77                  return;
78              }
79              if(this.changes === 0) {
80                  // No rows were updated.
81                  return res.status(404).send('Student record does not exist!');
82              }
83              res.status(200).send('Student record updated successfully!');
84          });
85      });
86

```

e) DELETE /students/:id: Delete a student record.

Below screenshot from app\app.js shows the 'DELETE' endpoint to delete a record of existing student by id.

```

86
87  /**** DELETE endpoint - Delete a student record. */
88  app.delete('/students/:id', function(req,res){
89      db.run(`Delete from students where id = ?`,
90          [
91              req.params.id
92          ],
93          function(err){
94              if(err){
95                  res.status(400).send('Error deleting Student record!');
96                  return;
97              }
98              if(this.changes === 0) {
99                  // No rows were updated.
100                  return res.status(404).send('Student record does not exist!');
101              }
102              res.status(200).send('Student record deleted successfully');
103          }
104      );
105  });
106

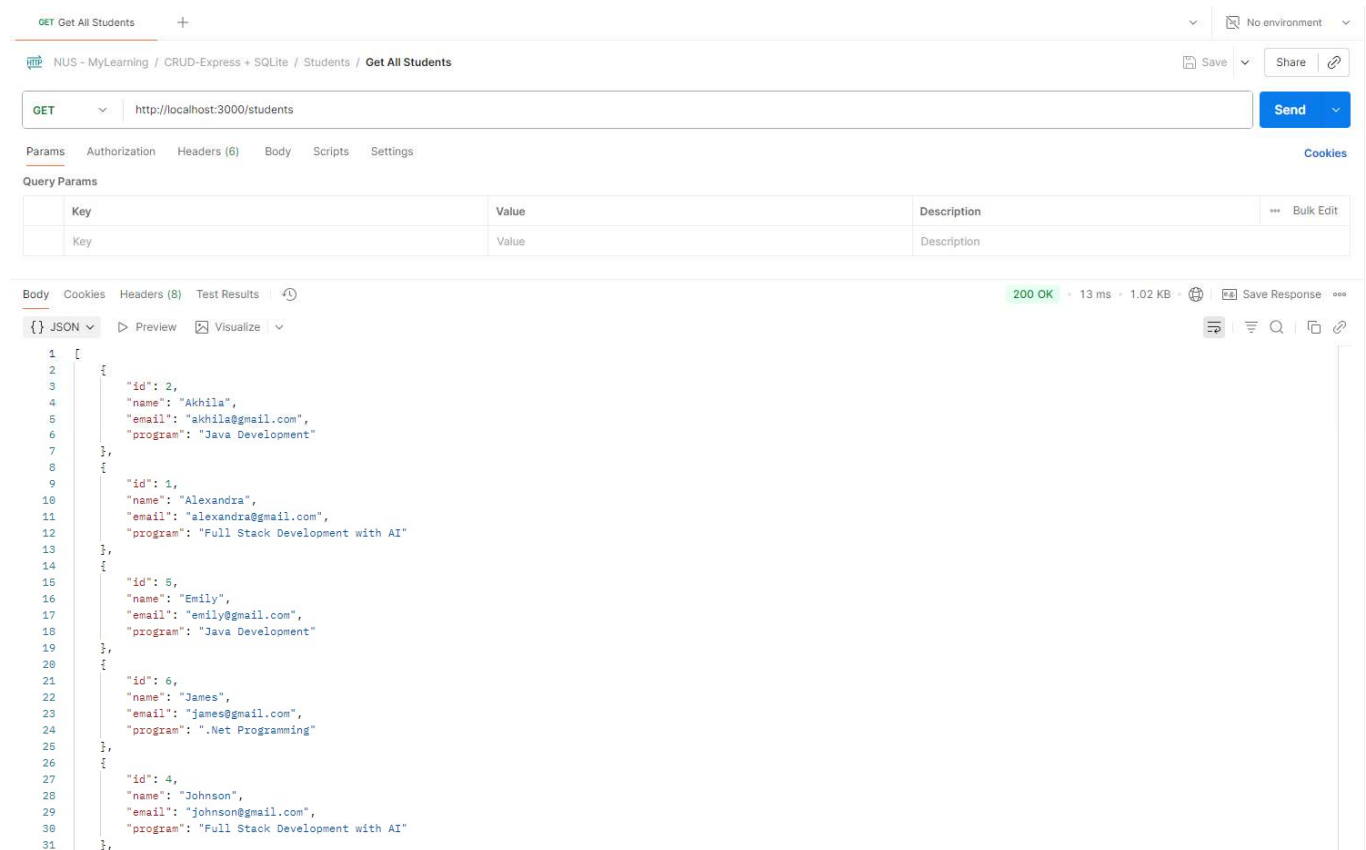
```

Task 2: Test the API with Postman

Please find below verification of endpoints done using Postman.

a) Verify → GET /students: Retrieve a list of all students.

Below screenshot shows all the student records are fetched and status code is 200.



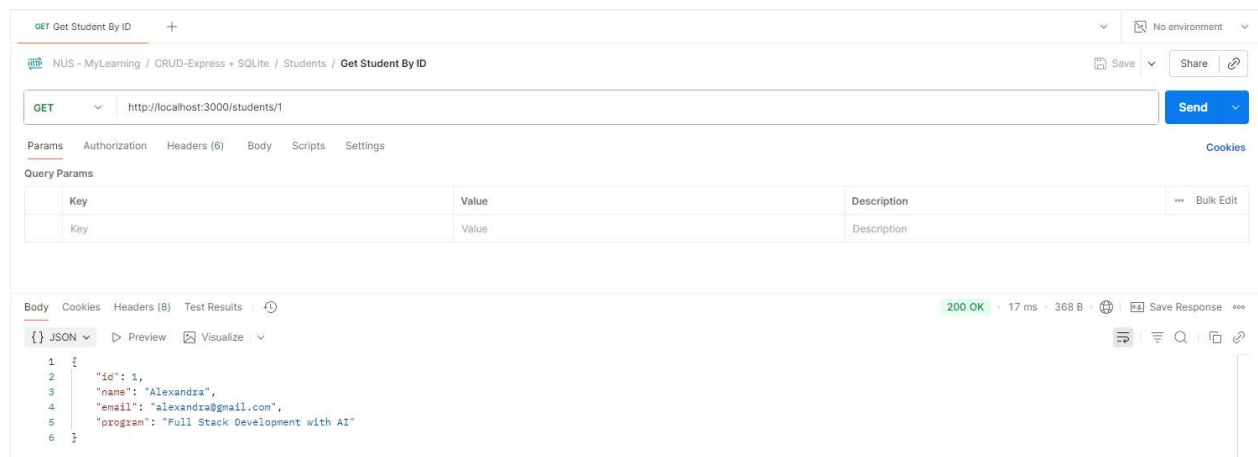
The screenshot shows a Postman test for the GET /students endpoint. The URL is http://localhost:3000/students. The response status is 200 OK, with a response time of 13 ms and a body size of 1.02 KB. The response is displayed in JSON format, showing a list of 5 student records.

Key	Value	Description
Key	Value	Description

```
1 [
2   {
3     "id": 2,
4     "name": "Akhila",
5     "email": "akhila@gmail.com",
6     "program": "Java Development"
7   },
8   {
9     "id": 1,
10    "name": "Alexandra",
11    "email": "alexandra@gmail.com",
12    "program": "Full Stack Development with AI"
13  },
14  {
15    "id": 5,
16    "name": "Emily",
17    "email": "emily@gmail.com",
18    "program": "Java Development"
19  },
20  {
21    "id": 6,
22    "name": "James",
23    "email": "james@gmail.com",
24    "program": ".Net Programming"
25  },
26  {
27    "id": 4,
28    "name": "Johnson",
29    "email": "johnson@gmail.com",
30    "program": "Full Stack Development with AI"
31  }
32 ]
```

b) Verify → GET /students/:id: Retrieve a specific student by its ID.

Below screenshot shows the student record is retrieved correctly using id. The status code is 200.

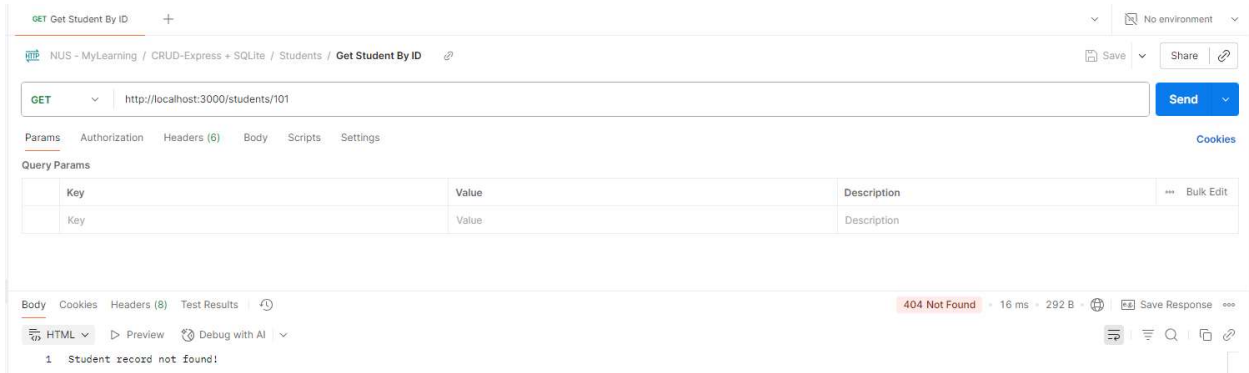


The screenshot shows a Postman test for the GET /students/:id endpoint. The URL is http://localhost:3000/students/1. The response status is 200 OK, with a response time of 17 ms and a body size of 368 B. The response is displayed in JSON format, showing the details of the student with ID 1.

Key	Value	Description
Key	Value	Description

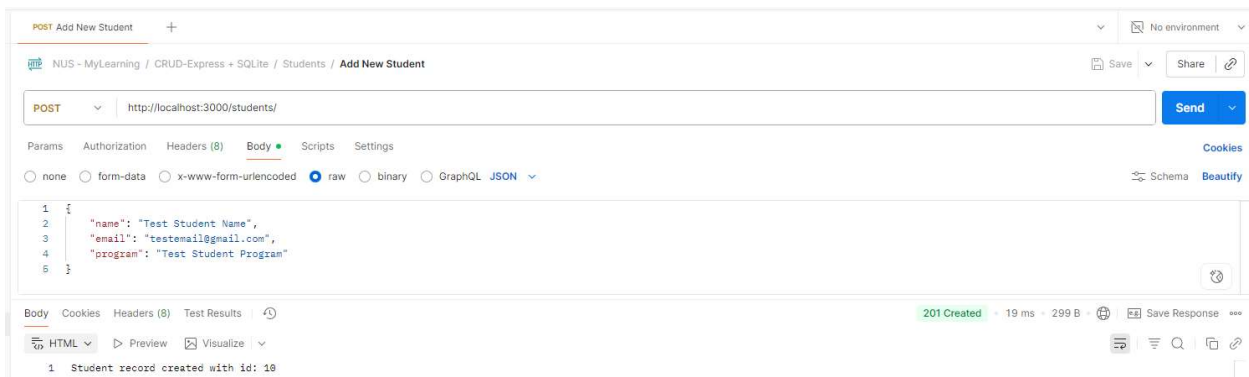
```
1 {
2   "id": 1,
3   "name": "Alexandra",
4   "email": "alexandra@gmail.com",
5   "program": "Full Stack Development with AI"
6 }
```

Below screenshot shows, the student record with id=101 does not exist. The status code returned is 404.

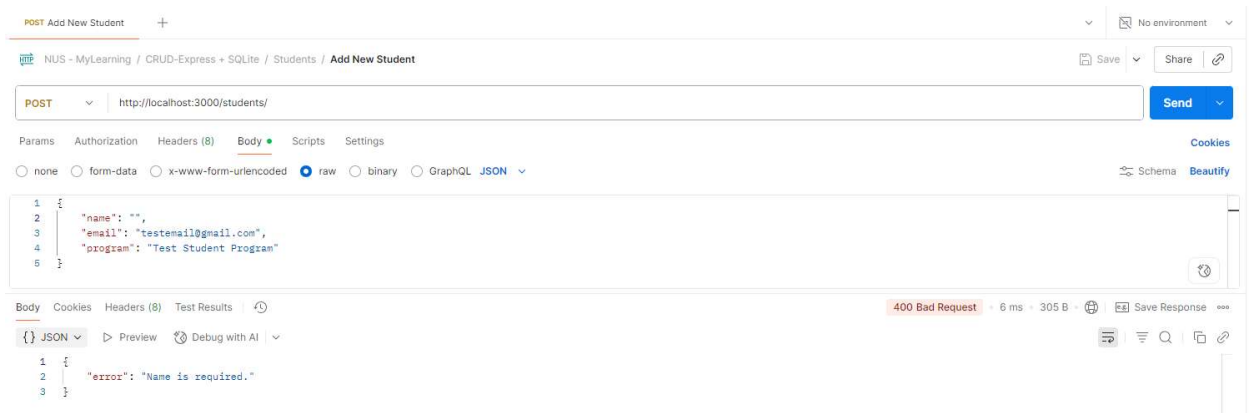


c) **Verify → POST /students: Add a new student.**

Below screenshot shows, the new Student record is added successfully. The status code returned is 201.

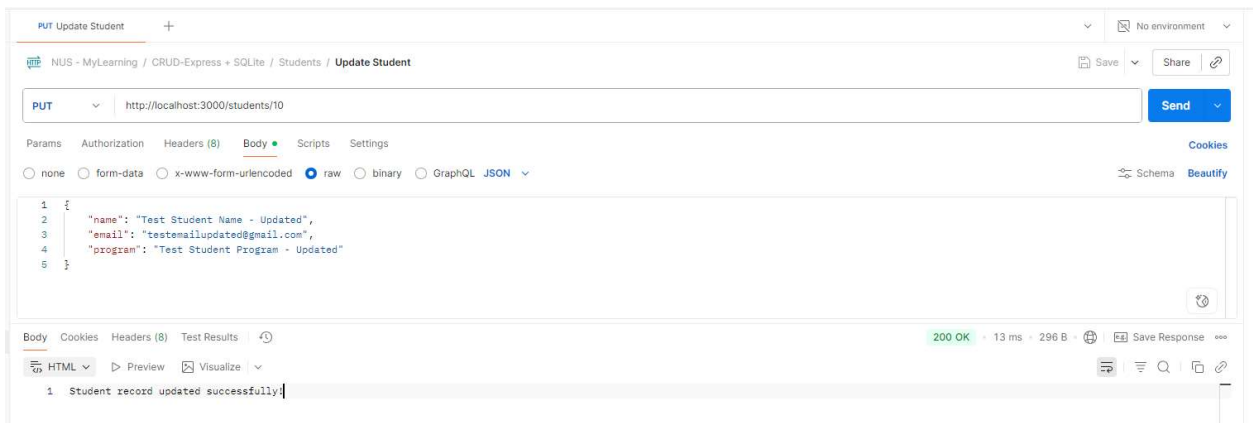


The below screenshot shows the API returned status code 400, as the required value for 'name' field is not provided.

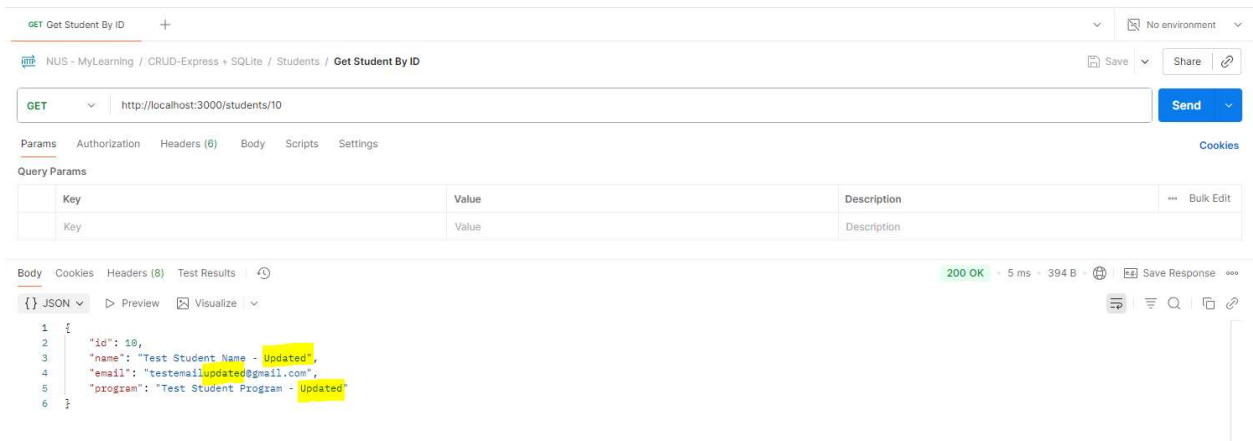


d) **Verify → PUT /students/:id: Update an existing student.**

Below screenshot shows, the Student record is updated successfully. The status code returned is 200.

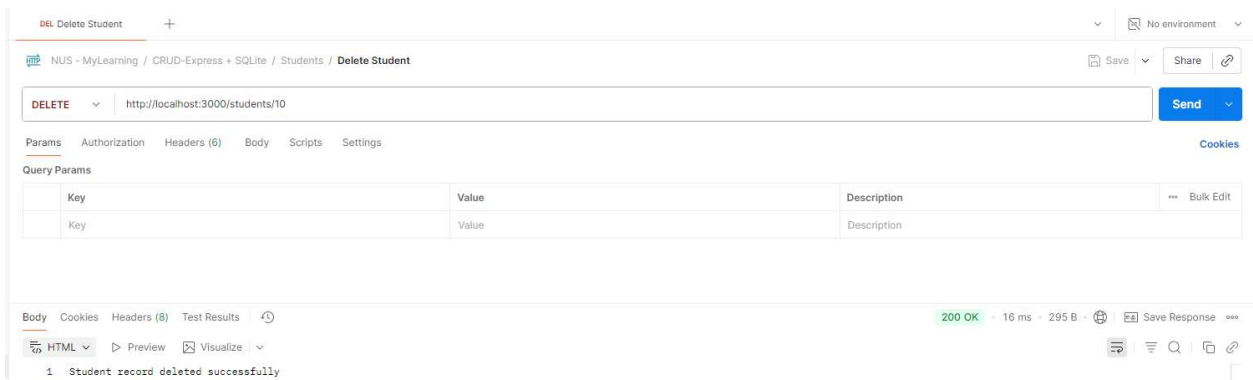


Use GET method by id to verify the changes saved to database.

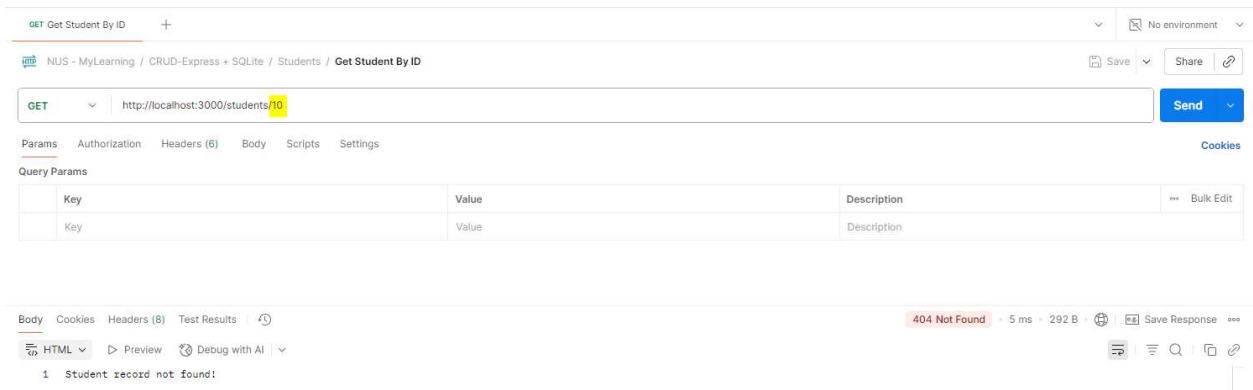


e) Verify → DELETE /students/:id: Delete a student record.

Below screenshot shows, the Student record is deleted successfully. The status code returned is 200.



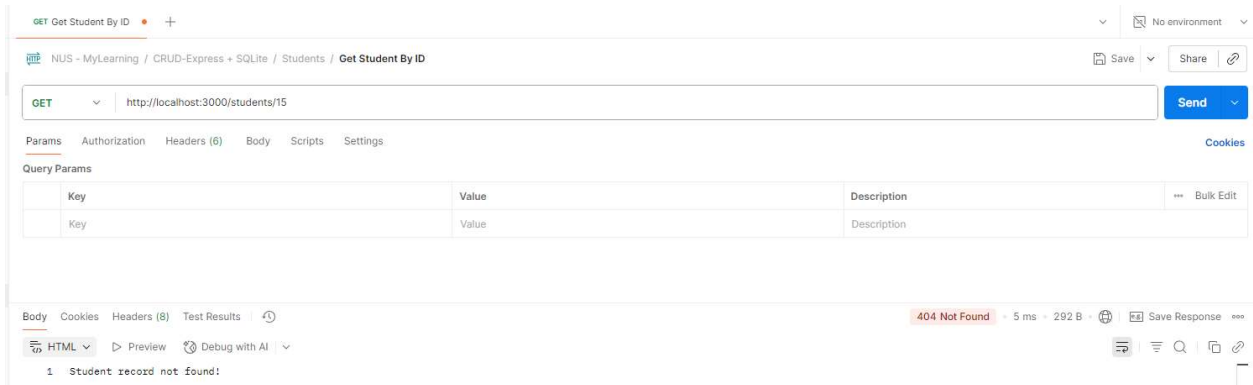
Use GET method by id to verify that the record is deleted and does not exist in database. The API returns status code 404 (Not found).



Task 3: Error Handling and Input Validation

a) If a student record is not found in the GET, PUT, or DELETE requests, return a **404 Not Found** response.

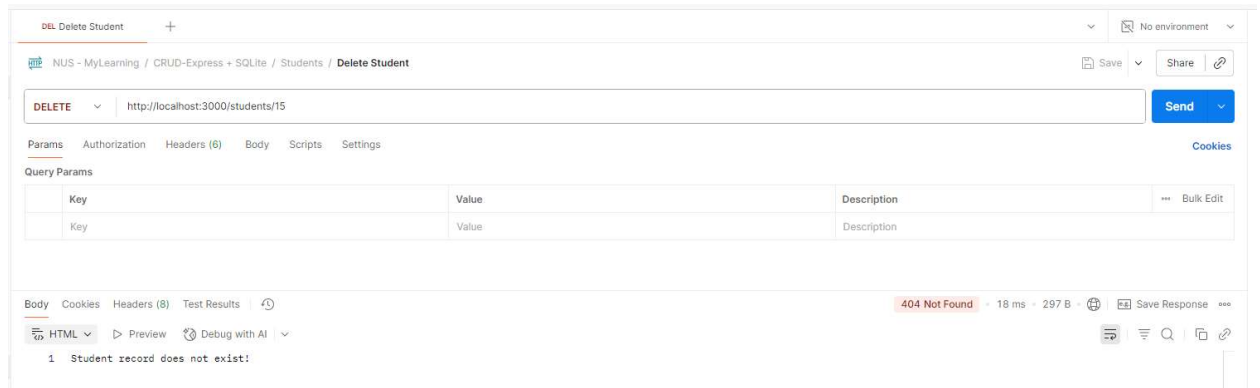
1) Below screenshot shows that status code 404 is returned when student record is not found in GET request.



2) Below screenshot shows that status code 404 is returned when student record is not found, in PUT request.

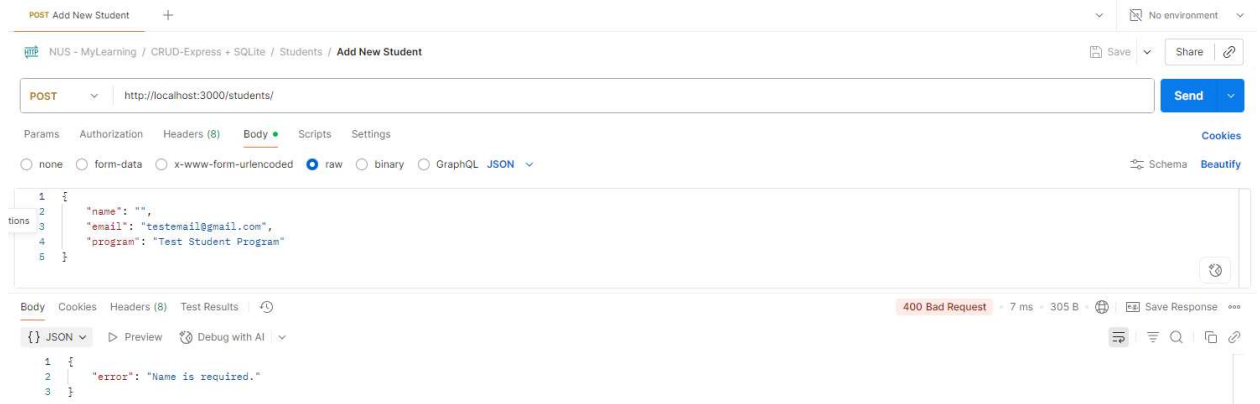


- 3) Below screenshot shows that status code 404 is returned when student record is not found, in DELETE request.



b) If invalid data is sent in the POST or PUT requests, return a 400 Bad Request response.

- 1) Below screenshot shows, the status code 400 (Bad Request) is returned from the POST request, when the required field name is not provided.



- 2) Below screenshot shows, the status code 400 (Bad Request) is returned from the PUT request, when the required field name is not provided.

